

UI Architectural Blueprint for WhisperLocal: A Modular, Production-Grade Implementation Strategy

1. Executive Summary and Strategic Context

The transition of the "WhisperLocal" application from a functional prototype to a production-grade desktop utility represents a significant architectural inflection point. The current state, as described in the "Research & Requirements Document," exhibits classic symptoms of "glue code" fatigue: disconnected UI elements, unresponsive interface controls ("dead buttons"), and a fragile synchronization between the frontend visual state and the backend logic. These are not merely bugs; they are structural deficits inherent in moving from a script-based paradigm to an event-driven graphical user interface (GUI) architecture.

This report provides a comprehensive, expert-level analysis and implementation strategy to resolve these disconnects. The objective is to design a system that is robust, modular, and scalable, strictly avoiding temporary "hacky" solutions in favor of industry-standard patterns. The core of this strategy revolves around transforming the pywebview bridge from a simple function passer into a structured **Hierarchical API**, implementing a **Task-Queue Concurrency Model** to manage heavy AI inference loads without blocking the UI, and establishing a **Single Source of Truth (SSOT)** for state management using strict schema validation.

The analysis draws upon extensive research into hybrid application architectures, Python concurrency models, and embedded database strategies. We identify that the primary friction points—latency during model loading, lack of VRAM awareness, and synchronization drift—stem from a monolithic approach to the API bridge. By refactoring this into a domain-oriented modular architecture, leveraging `ThreadPoolExecutor` for non-blocking I/O, and utilizing SQLite for high-performance time-series analytics, WhisperLocal can achieve the responsiveness and reliability expected of professional software.

1.1 The Shift to Hybrid Desktop Architectures

The architecture requested—wiring an HTML/JavaScript frontend to a Python backend—follows the modern "Hybrid Desktop" paradigm. Unlike traditional frameworks (Qt, Tkinter) where the UI and logic share the same memory space and thread management rules, pywebview introduces a distinct separation of concerns. The frontend operates in a browser engine (WebKit, Edge, or CEF), while the backend operates in a standard Python interpreter.

This separation offers immense flexibility but introduces the "Bridge Latency" and "Serialization Overhead" challenges that are currently manifesting as broken buttons and frozen UIs in the current build.

The "disconnects" identified in the research document are symptomatic of a failure to respect this boundary. The frontend assumes immediate execution of tasks, while the backend requires significant time to load neural networks (OpenAI's Whisper) into memory. The proposed solution bridges this temporal gap through asynchronous event handling and rigid state management protocols.

2. Architectural Pattern: The Modular Hierarchical Bridge

2.1 Limitations of the Monolithic API Pattern

In early-stage development with pywebview, a common pattern is the "God Object" or Monolithic API. This involves creating a single Python class (e.g., class Api:) and attaching every necessary method to it—mixing settings management, file I/O, AI inference, and window control into one namespace.²

While this approach is easy to scaffold, it creates significant technical debt:

1. **Cognitive Overload:** Developers must navigate a single file containing thousands of lines of unrelated logic.
2. **Namespace Pollution:** To avoid collisions, method names become verbose (e.g., save_settings_to_file, save_transcription_result), which leads to brittle JavaScript calls.
3. **Tight Coupling:** Changing the logic for model loading might inadvertently break the logic for saving settings if they share internal state variables within the massive class instance.
4. **Testing Bottlenecks:** Unit testing a monolithic class requires mocking the entire application state rather than isolated components.²

As detailed in research on modular architectures, monoliths shine in MVPs but become "dangerous" as applications scale, making refactoring difficult and slowing down testing cycles.²

2.2 Proposed Solution: Domain-Oriented Nested API

To address the requirement for a "clean pattern," we propose utilizing the pywebview capability to expose nested classes.³ This allows us to structure the backend into distinct controllers (modules) while maintaining a single entry point for the frontend.

The architecture will be composed of a root AppApi class that initializes and exposes three

distinct sub-controllers:

- **SettingsController:** Exclusively handles the user_settings.json lifecycle—loading, validating (via Pydantic), and saving configuration data.
- **TranscriptionController:** Manages the Whisper model lifecycle. This includes VRAM checks, model loading/unloading, and the transcription loop itself.
- **SystemController:** Handles OS-level interactions such as window management, file dialogs, and usage statistics retrieval from the database.

2.2.1 JavaScript Representation and Ergonomics

This hierarchical structure translates directly into a namespaced JavaScript object, significantly improving code readability and maintainability on the frontend.

Feature	Monolithic Approach (Current)	Modular Approach (Proposed)
JS Call	pywebview.api.load_model_large()	pywebview.api.transcription.load('large')
JS Call	pywebview.api.save_user_settings(data)	pywebview.api.settings.save(data)
State	Shared self.current_model mixed with self.theme	Isolated self.transcription.current_model
Scalability	Linear complexity increase	Logarithmic complexity increase

This structure mirrors the "Domain Module API" pattern described in modular monolith literature, where each module encapsulates its implementation details behind a stable façade.⁴

2.3 Implementation Strategy for Nested Classes

To implement this in pywebview, the root API class acts as a container. Crucially, pywebview converts nested Python objects into nested JavaScript objects automatically, provided they are public attributes.³

Python

```
class AppApi:
    def __init__(self, window):
        self.settings = SettingsController(window)
        self.transcription = TranscriptionController(window)
        self.system = SystemController(window)
```

The "Wiring" requirement for the frontend is satisfied by this structure. The JavaScript "dead buttons" currently looking for nonexistent functions can be re-bound to these namespaced methods. This enforces a separation of concerns: the TranscriptionController does not need to know about the SettingsController directly; it only needs to accept configuration parameters, making the code loosely coupled.⁴

3. Concurrency and Latency: Solving the "Frozen UI"

3.1 The Physics of Blocking I/O in GUI Loops

The most critical user experience (UX) flaw identified in the research document is the "freezing" of the UI during long-running tasks like model loading. This occurs because pywebview relies on a GUI event loop (Cocoa on macOS, WinForms on Windows, GTK on Linux) to process user inputs and repaint the window.

When the JavaScript frontend calls a Python function via pywebview.api, that function is executed. If the function performs a synchronous, blocking operation—such as `whisper.load_model("large")`, which involves reading gigabytes from disk and allocating GPU memory—the Python interpreter blocks. Consequently, the GUI loop cannot cycle, the window stops responding to OS events, and the operating system eventually flags the application as "Not Responding".⁵

3.2 The Concurrency Model: Threading vs. Asyncio

To solve this, the blocking work must be offloaded from the main thread. We analyzed two primary approaches: asyncio and threading.

3.2.1 Analysis of Asyncio

Python's asyncio library is excellent for network-bound I/O (e.g., waiting for HTTP responses). However, it has significant limitations in this specific context:

- **CPU-Bound Tasks:** The Whisper model initialization is CPU and bus-bound (loading weights). asyncio event loops are blocked by CPU-intensive tasks unless explicitly run in a

separate executor.

- **Integration Complexity:** Integrating an asyncio loop with the existing GUI loop of pywebview (which manages its own loop) is complex and prone to race conditions or "loop already running" errors.⁶
- **Library Support:** While some libraries are async-native, the standard openai-whisper library is synchronous. Wrapping it requires `run_in_executor` regardless, negating the native syntactic benefits of `async/await` in Python.⁸

3.2.2 The Chosen Solution: Task Queue with ThreadPoolExecutor

We recommend using the standard threading module via `concurrent.futures.ThreadPoolExecutor`. This is the robust, production-grade solution for handling mixed I/O and CPU tasks in Python GUI applications.⁵

- **GIL Considerations:** Python's Global Interpreter Lock (GIL) prevents multiple threads from executing Python bytecode simultaneously. However, highly optimized libraries like torch (which Whisper uses) and numpy frequently release the GIL during heavy C/C++ operations (like matrix multiplication or moving tensors to VRAM). This means a background thread loading the model *will* run truly in parallel with the main thread, keeping the UI responsive.⁹

3.3 The "Fire-and-Forget" Pattern with Feedback

To wire the "Model Selection" buttons effectively without freezing the UI, we must adopt a specific pattern:

1. **Frontend Trigger:** User clicks "Load Large Model."
2. **API Call:** JS calls `api.transcription.load_model('large')`.
3. **Main Thread (Python):**
 - Accepts the request.
 - Checks if a task is already running (locking).
 - Submits the actual loading work to the `ThreadPoolExecutor`.
 - **Immediately returns** a "Request Accepted" status to the UI.
4. **Worker Thread (Python):**
 - Performs the heavy lifting (VRAM check, Disk I/O, Model Load).
 - **Callback:** Once finished, it invokes `window.evaluate_js()` to notify the UI of success or failure.

This disconnects the *request* for work from the *execution* of work, ensuring the UI thread never waits for the model.¹⁰

4. State Management: Unidirectional Data Flow

4.1 The Synchronization Disconnect

The research document highlights a disconnect where the UI state does not match the backend settings, especially after a restart. This is a classic "Split-Brain" problem where two sources of truth exist: the DOM (HTML inputs) and the `user_settings.json` file.

4.2 Single Source of Truth (SSOT) Strategy

To resolve this, we must strictly enforce that **Python is the Single Source of Truth**. The HTML UI is merely a "projection" or "view" of the Python state. The UI should never update its own state effectively until the backend confirms the change.

4.2.1 The Cycle of State

We propose a unidirectional flow pattern, inspired by patterns like Redux but adapted for the Pywebview bridge:

1. **Boot Phase:** On application startup, Python loads `user_settings.json` via a Pydantic model (ensuring types are correct).
2. **Hydration Phase:** When the frontend loads, it fires a `pywebviewready` event. The first action of the JS Bridge is to call `api.settings.get_all()`.
3. **Rendering:** The frontend receives the JSON object and populates all inputs/checkboxes. The UI is now synchronized.
4. **Update Phase:**
 - User toggles "Use GPU" in the UI.
 - **Constraint:** The UI does *not* immediately rely on the checkbox state.
 - **Action:** JS sends `api.settings.update({ use_gpu: true })`.
 - **Validation:** Python validates the request. If valid, it writes to disk.
 - **Propagation:** Python broadcasts a `settings_updated` event back to the UI.
 - **Confirmation:** The UI updates the visual indicator based on the broadcast.

This cycle prevents invalid states (e.g., selecting a non-existent path) from ever persisting in the UI, as the backend validation occurs *before* the visual confirmation.³

5. Data Persistence and Analytics Architecture

5.1 Requirement Analysis: Graphs and Usage Stats

The requirement to "Propose a data structure for storing usage stats that allows for fast retrieval and rendering in Chart.js" presents a choice between simple file-based storage (JSON) and a relational database (SQLite).

5.2 Technology Selection: SQLite vs. JSON

While JSON is human-readable and easy to implement for configuration settings, it is

mathematically ill-suited for time-series logs that grow indefinitely.¹³

Feature	JSON Storage	SQLite Storage
Read Complexity	$O(N)$ - Must parse entire file to read one record.	$O(\log N)$ - B-Tree indexing allows instant retrieval.
Aggregation	Manual looping in Python/JS. Slow.	Native GROUP BY, SUM(), AVG(). Fast.
Concurrency	File locking issues on write.	WAL mode supports concurrent readers/writers.
Data Integrity	Prone to corruption if app crashes during write.	ACID compliant atomic transactions.

Decision: We will use **SQLite** for usage statistics. The overhead is negligible (built into Python), and it enables complex queries (e.g., "Total minutes transcribed last month by model type") to be executed in milliseconds, which is critical for rendering responsive charts.¹⁴

5.3 Data Structure for Chart.js

Chart.js requires data to be formatted as arrays of labels and values.¹⁶ To minimize frontend processing, the Python backend should return data already transformed into this structure.

Proposed Schema (usage_stats.db):

SQL

```
CREATE TABLE IF NOT EXISTS transcription_logs (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,  
  duration_seconds REAL,  
  model_name TEXT,  
  status TEXT,  
  input_tokens INTEGER  
);
```

Optimization for Graphs:

Instead of sending 10,000 raw rows to JavaScript (which would cause lag), the backend will expose an endpoint `get_daily_usage(days=30)` that executes an aggregation query:

SQL

```
SELECT
  DATE(timestamp) as log_date,
  SUM(duration_seconds) as total_seconds
FROM transcription_logs
WHERE timestamp >= date('now', '-30 days')
GROUP BY log_date
ORDER BY log_date ASC;
```

This returns exactly the data needed for the X-axis (Dates) and Y-axis (Seconds), satisfying the constraint for "fast retrieval and rendering".¹⁸

6. Implementation Specification: The Python Backend

This section provides the "Skeleton code block" requested, integrating the Modular API, Threading, Pydantic Settings, and SQLite logic.

6.1 Settings Definition (Pydantic)

We use Pydantic to define the JSON schema. This ensures that if the user manually edits the `user_settings.json` file with invalid data (e.g., `"beam_size": "five"` instead of 5), the app detects it on startup.²⁰

Python

```
import json
import os
import threading
import sqlite3
```



```

import time
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path
from typing import Optional, Dict, Any

from pydantic import BaseModel, Field, ValidationError, validator
import webview
import torch
# import whisper # Commented out for mock prototyping

# --- 1. Settings Schema ---
class TranscriptionSettings(BaseModel):
    model_size: str = Field("base", pattern="^(tiny|base|small|medium|large)$")
    language: Optional[str] = Field(None, min_length=2, max_length=2)
    use_gpu: bool = True

class AppSettings(BaseModel):
    save_location: str = Field(str(Path.home() / "Documents" / "WhisperLocal"))
    theme: str = Field("system", pattern="^(system|light|dark)$")
    transcription: TranscriptionSettings = Field(default_factory=TranscriptionSettings)

    @validator('save_location')
    def validate_path(cls, v):
        path = Path(v)
        if not path.exists():
            # Auto-create if missing to ensure robustness
            try:
                path.mkdir(parents=True, exist_ok=True)
            except OSError:
                raise ValueError(f"Cannot create directory: {v}")
        return str(path)

# --- 2. Modular Controllers ---

class SettingsController:
    """Manages configuration state and persistence."""
    def __init__(self, window):
        self.window = window
        self.file_path = "user_settings.json"
        self.model = self._load_from_disk()

    def _load_from_disk(self) -> AppSettings:
        if not os.path.exists(self.file_path):

```

```
    return AppSettings()
```

```
    try:
```

```
        with open(self.file_path, 'r') as f:
```

```
            data = json.load(f)
```

```
        return AppSettings(**data)
```

```
    except (json.JSONDecodeError, ValidationError) as e:
```

```
        print(f"Settings corruption detected: {e}. Reverting to defaults.")
```

```
    return AppSettings()
```

```
def get_all(self) -> Dict[str, Any]:
```

```
    """Exposed to JS: Returns current settings."""
```

```
    return self.model.model_dump()
```

```
def save(self, data: Dict[str, Any]) -> Dict[str, Any]:
```

```
    """Exposed to JS: Validates and saves settings."""
```

```
    try:
```

```
        # Update internal model with validation
```

```
        new_settings = AppSettings(**data)
```

```
        self.model = new_settings
```

```
    # Persist to disk
```

```
    with open(self.file_path, 'w') as f:
```

```
        f.write(self.model.model_dump_json(indent=2))
```

```
    # Notify UI of success
```

```
    return {"success": True, "settings": self.model.model_dump()}
```

```
    except ValidationError as e:
```

```
        return {"success": False, "error": str(e)}
```

```
class TranscriptionController:
```

```
    """Manages AI Model lifecycle and heavy lifting."""
```

```
    def __init__(self, window):
```

```
        self.window = window
```

```
        self.current_model_name = None
```

```
        self.is_loading = False
```

```
        # Thread pool for non-blocking operations
```

```
        self.executor = ThreadPoolExecutor(max_workers=1)
```

```
    def load_model(self, model_name: str):
```

```
        """Exposed to JS: Initiates background model loading."""
```

```
        if self.is_loading:
```

```
            return {"status": "error", "message": "A model is already loading."}
```

```

self.is_loading = True
# Submit task to background thread
self.executor.submit(self._load_model_task, model_name)
return {"status": "pending", "message": f"Queued load for {model_name}..."}

```

```

def _load_model_task(self, model_name: str):
    """Internal: Runs on worker thread."""
    try:
        # 1. Edge Case: VRAM Check
        self._check_vram_requirements(model_name)

        # 2. Simulate Loading Progress (Mocking whisper.load_model)
        for progress in range(0, 101, 20):
            time.sleep(0.5) # Simulate heavy I/O
            # Thread-safe UI update via evaluate_js
            self.window.evaluate_js(f"window.bridgeEvents.onLoadProgress({progress})")

        # 3. Success State
        self.current_model_name = model_name
        self.is_loading = False
        self.window.evaluate_js(f"window.bridgeEvents.onModelLoaded('{model_name}')")

```

```

except RuntimeError as e:
    self.is_loading = False
    # Bubble error to UI
    safe_error = str(e).replace("\n", "")
    self.window.evaluate_js(f"window.bridgeEvents.onError('{safe_error}')")

```

```

def _check_vram_requirements(self, model_name: str):
    """Heuristic VRAM check using torch/pynvml."""
    if not torch.cuda.is_available():
        return # CPU mode is slow but safe from VRAM crashes

```

```

# Approximate VRAM reqs (GB)
requirements = {'tiny': 1, 'base': 1, 'small': 2, 'medium': 5, 'large': 10}
req_gb = requirements.get(model_name, 1)

```

```

# Get free memory
free_bytes = torch.cuda.mem_get_info()
free_gb = free_bytes / (1024 ** 3)

```

```

if free_gb < req_gb:

```

```
        raise RuntimeError(f"Insufficient VRAM. {model_name.title()} requires ~{req_gb}GB, but you  
have {free_gb:.1f}GB.")
```

```
class StatsController:
```

```
    """Manages SQLite analytics."""
```

```
    def __init__(self, window):
```

```
        self.window = window
```

```
        self.db_path = "usage_stats.db"
```

```
        self._init_db()
```

```
    def _init_db(self):
```

```
        conn = sqlite3.connect(self.db_path)
```

```
        cursor = conn.cursor()
```

```
        cursor.execute("""
```

```
            CREATE TABLE IF NOT EXISTS usage (
```

```
                id INTEGER PRIMARY KEY,
```

```
                date TEXT,
```

```
                duration REAL
```

```
            )
```

```
        """)
```

```
        conn.commit()
```

```
        conn.close()
```

```
    def get_chart_data(self):
```

```
        """Exposed to JS: Returns formatted data for Chart.js."""
```

```
        conn = sqlite3.connect(self.db_path)
```

```
        cursor = conn.cursor()
```

```
        # Aggregation for graph
```

```
        cursor.execute("""
```

```
            SELECT date, SUM(duration)
```

```
            FROM usage
```

```
            GROUP BY date
```

```
            ORDER BY date DESC
```

```
            LIMIT 7
```

```
        """)
```

```
        rows = cursor.fetchall()
```

```
        conn.close()
```

```
        # Format for Chart.js
```

```
        labels = [r for r in rows][::-1]
```

```
        data = [r for r in rows][::-1]
```

```
        return {"labels": labels, "datasets": [{"data": data}]}
```

```

# --- 3. Root API ---
class AppApi:
    """Root API exposed to JavaScript."""
    def __init__(self):
        self.window = None
        # Sub-controllers initialized to None until window is created
        self.settings = None
        self.transcription = None
        self.stats = None

    def set_window(self, window):
        """Called after webview.create_window"""
        self.window = window
        self.settings = SettingsController(window)
        self.transcription = TranscriptionController(window)
        self.stats = StatsController(window)

# --- Entry Point ---
def start_app():
    api = AppApi()
    window = webview.create_window(
        title="WhisperLocal Pro",
        url="index.html",
        js_api=api,
        width=1000,
        height=800
    )
    api.set_window(window)
    webview.start(debug=True)

if __name__ == "__main__":
    start_app()

```

7. Implementation Specification: The JavaScript Frontend

The frontend "Wiring" solution requires a robust bridge function to handle the Promises returned by pywebview. It must also establish the event listeners that the Python backend will trigger.

7.1 The Bridge Abstraction

We avoid calling `pywebview.api` directly in event listeners. Instead, we create a wrapper Bridge object that handles error catching and logging globally.

JavaScript

```
/**
 * bridge.js - The Frontend/Backend Interconnect
 */

const Bridge = {
  // State buffer to prevent UI thrashing
  state: {
    isTranscribing: false,
    currentModel: null
  },

  /**
   * Generic wrapper for Python calls.
   * Handles the Promise chain and global error bubbling.
   */
  async call(apiMethod,...args) {
    if (!window.pywebview) {
      console.warn("Pywebview not ready. Call ignored:", apiMethod);
      return null;
    }
    try {
      return await apiMethod(...args);
    } catch (error) {
      console.error("Bridge Error:", error);
      ui.showToast(`System Error: ${error}`, "error");
      throw error;
    }
  },

  // --- Module Wrappers ---

  settings: {
    async load() {
      // "Hydrate" the form
    }
  }
}
```

```

        const data = await Bridge.call(pywebview.api.settings.get_all);
        if (data) ui.forms.populate(data);
    },
    async save(formData) {
        const result = await Bridge.call(pywebview.api.settings.save, formData);
        if (result.success) {
            ui.showToast("Settings Saved", "success");
        } else {
            ui.showModal("Validation Error", result.error);
        }
    }
},

transcription: {
    async selectModel(modelName) {
        ui.setLoading(true, "Initializing Model...");
        // Returns immediately; completion handled via events
        const response = await Bridge.call(pywebview.api.transcription.load_model,
modelName);
        console.log("Model Load Requested:", response);
    }
},

stats: {
    async refreshChart() {
        const chartData = await Bridge.call(pywebview.api.stats.get_chart_data);
        ui.charts.renderUsage(chartData);
    }
}
};

// --- Event Listeners (Called from Python) ---

window.bridgeEvents = {
    onLoadProgress: (percent) => {
        document.getElementById('progressBar').style.width = `${percent}%`;
        document.getElementById('statusText').innerText = `Loading... ${percent}%`;
    },

    onModelLoaded: (modelName) => {
        ui.setLoading(false);
        Bridge.state.currentModel = modelName;
        ui.showToast(`Model ${modelName} ready`, "success");
    }
};

```

```

// Update "Dead Buttons" visual state
document.querySelectorAll('.model-btn').forEach(btn => btn.classList.remove('active'));
document.getElementById(`btn-${modelName}`).classList.add('active');
},

onError: (msg) => {
  ui.setLoading(false);
  ui.showModal("Critical Error", msg);
}
};

// --- Initialization ---
window.addEventListener('pywebviewready', () => {
  console.log("Bridge Established.");
  Bridge.settings.load();
  Bridge.stats.refreshChart();
});

```

8. Critical Edge Cases and Robustness

8.1 VRAM Resource Management

A major risk in local AI applications is hardware resource exhaustion. The "Large" Whisper model requires ~10GB of VRAM. If a user on a standard laptop (often sharing system RAM with the GPU) attempts to load this, the application could crash silently or freeze the OS.

Solution Logic:

The TranscriptionController in the Python prototype includes a specific method `_check_vram_requirements`.

1. It utilizes `torch.cuda.mem_get_info()` (or `pynvml` for more granular NVIDIA control) to check available memory.⁹
2. It compares this against a predefined requirement map.
3. **Error Bubbling:** If resources are insufficient, it raises a controlled exception. This exception is caught in the worker thread and sent to the UI via `window.bridgeEvents.onError`.
4. **UX Outcome:** The user sees a modal: *"Insufficient VRAM. Large requires 10GB, you have 4GB. Please try Medium or Small."* The app remains responsive.

8.2 Application Lifecycle and Graceful Shutdown

Long-running threads (like transcription) can prevent the Python process from terminating

when the window is closed, leaving "zombie" processes.

Implementation:

1. Use `threading.Event` to signal a stop condition.
2. Bind `window.events.closed` in `pywebview` to trigger a cleanup method in the API.
3. The API sets the stop flag.
4. The transcription loop checks this flag after every segment: `if self.stop_event.is_set(): break`.
5. This ensures database connections are closed and threads join cleanly before the process exits.²²

8.3 Error Propagation Strategy

Errors in the backend (Python) must be translated into user-intelligible messages in the frontend (JS).

- **Synchronous Errors** (e.g., Settings Validation): Returned directly in the return value of the Promise (`{ success: false, error: "..." }`).
- **Asynchronous Errors** (e.g., Model Load Failure): Sent via the event system (`bridgeEvents.onError`). This dual strategy ensures that immediate feedback is handled via `await`, while background failures interrupt the user workflow appropriately via `modals`.²⁴

9. Conclusion

The transformation of `WhisperLocal` from its current disconnected state to a production-grade application requires a fundamental shift in architecture. The adoption of a **Modular Hierarchical API** solves the code maintainability crisis by respecting separation of concerns. The **Task-Queue Concurrency Model** utilizing `ThreadPoolExecutor` solves the "Frozen UI" latency issues by respecting the physics of blocking I/O. Finally, the move to **SQLite** and **Pydantic** solves the data integrity and analytics performance issues by utilizing tools designed for structure and aggregation.

This blueprint provides the necessary "wiring" to connect the frontend and backend effectively, fulfilling the requirements for a robust, scalable, and user-centric desktop application. The provided code prototypes serve as the foundational skeleton for this refactoring effort, ensuring that every button click triggers a predictable, safe, and responsive action in the system.

Works cited

1. Monolithic vs Modular Architecture in Mobile Apps: A Deep Dive for Scalable Development, accessed February 10, 2026, <https://openmobilekit.medium.com/monolithic-vs-modular-architecture-in-mobil>

- [e-apps-a-deep-dive-for-scalable-development-6e77d1766225](#)
2. Javascript–Python bridge - pywebview - Example, accessed February 10, 2026, <https://pywebview.flowrl.com/guide/interdomain>
 3. Architectural patterns for modular monoliths that enable fast flow - Microservices.io, accessed February 10, 2026, <https://microservices.io/post/architecture/2024/09/09/modular-monolith-patterns-for-fast-flow.html>
 4. What's the Best Way to Handle Concurrency in Python: ThreadPoolExecutor or asyncio? | by Soumit Salman Rahman | Towards Dev - Medium, accessed February 10, 2026, <https://medium.com/towardsdev/whats-the-best-way-to-handle-concurrency-in-python-threadpoolexecutor-or-asyncio-85da1be58557>
 5. python - asyncio or threads for long running background tasks - Stack Overflow, accessed February 10, 2026, <https://stackoverflow.com/questions/75124041/asyncio-or-threads-for-long-running-background-tasks>
 6. How would you handle this asynchronous programming issue? : r/learnpython - Reddit, accessed February 10, 2026, https://www.reddit.com/r/learnpython/comments/8utm4s/how_would_you_handle_this_asynchronous/
 7. Proper way of running long sync task in python asyncio - Stack Overflow, accessed February 10, 2026, <https://stackoverflow.com/questions/76980079/proper-way-of-running-long-sync-task-in-python-asyncio>
 8. Get total amount of free GPU memory and available using pytorch - Stack Overflow, accessed February 10, 2026, <https://stackoverflow.com/questions/58216000/get-total-amount-of-free-gpu-memory-and-available-using-pytorch>
 9. Usage | pywebview - Example, accessed February 10, 2026, <https://pywebview.flowrl.com/guide/usage>
 10. What are thread-safety guarantees for javascript code · Issue #257 · r0x0r/pywebview, accessed February 10, 2026, <https://github.com/r0x0r/pywebview/issues/257>
 11. State sharing between Python and Javascript · Issue #1594 · r0x0r/pywebview - GitHub, accessed February 10, 2026, <https://github.com/r0x0r/pywebview/issues/1594>
 12. SQLite vs JSON - Which Data Processor Is Better In 2025? (FULL COMPARISON!), accessed February 10, 2026, https://www.youtube.com/watch?v=FP_o9Af0W14
 13. Using a database (such as sqlite3) versus JSON to store data? : r/flask - Reddit, accessed February 10, 2026, https://www.reddit.com/r/flask/comments/lvccgo/using_a_database_such_as_sqlite3_vs_json_to/
 14. SQLite or JSON? : r/learnprogramming - Reddit, accessed February 10, 2026, https://www.reddit.com/r/learnprogramming/comments/6ksvii/sqlite_or_json/
 15. Time Cartesian Axis | Chart.js, accessed February 10, 2026,

- <https://www.chartjs.org/docs/latest/axes/cartesian/time.html>
16. Data structures - Chart.js, accessed February 10, 2026,
<https://www.chartjs.org/docs/latest/general/data-structures.html>
 17. Time series chart with grouped by hour - Ignition - Inductive Automation Forum, accessed February 10, 2026,
<https://forum.inductiveautomation.com/t/time-series-chart-with-grouped-by-hour/65542>
 18. sqlite - SQL Aggregation by Hour - Database Administrators Stack Exchange, accessed February 10, 2026,
<https://dba.stackexchange.com/questions/311685/sql-aggregation-by-hour>
 19. JSON Schema - Pydantic Validation, accessed February 10, 2026,
https://docs.pydantic.dev/latest/api/json_schema/
 20. Pydantic: Simplifying Data Validation in Python, accessed February 10, 2026,
<https://realpython.com/python-pydantic/>
 21. Reload/Refresh a webpage using pywebview library in python - Stack Overflow, accessed February 10, 2026,
<https://stackoverflow.com/questions/63490363/reload-refresh-a-webpage-using-pywebview-library-in-python>
 22. How to Access pywebview.Window Object from Another multiprocessing.Process?, accessed February 10, 2026,
<https://stackoverflow.com/questions/69020223/how-to-access-pywebview-window-object-from-another-multiprocessing-process>
 23. Easier Error Handling Using Async/Await - Software, Fitness, and Gaming - Jesse Warden, accessed February 10, 2026,
<https://jessewarden.com/2017/11/easier-error-handling-using-asyncawait.html>
 24. Error handling in Async Await API calling - javascript - Stack Overflow, accessed February 10, 2026,
<https://stackoverflow.com/questions/68895938/error-handling-in-async-await-api-calling>